

TEMA: Conexión de Django con base de datos**Resultados de Aprendizaje:**

-  Que los estudiantes puedan gestionar la conectividad de MySQL con Django para la generación de aplicaciones
-  Que los estudiantes puedan generar modelo para la creación de tablas de la base de datos
-  Que los estudiantes comprendan los pasos para la generación de migraciones
-  Que los estudiantes tengan las primeras impresiones para la generación de consultas utilizando la consola interactiva de Django Shell

DESARROLLO

Requerimientos:

Para la realización de esta guía usted debe contar con los siguientes elementos

- Contar con el proyecto comercial generado en la guía anterior
- Contar con un servidor de base de datos mysql
- Crear una base de datos llamada **comercial**

Nota :

- Lamentablemente la base de datos que posee xampp no es compatible con la versión 6.1 de django por tanto deberá instalar la versión más reciente de mariadb server, esta la puede encontrar en <https://mariadb.org/download> y para que no sufra ningún conflicto con xamp utilice el puerto 3307
- Una vez instalado agregue al path de su sistema la dirección donde ha instalado mariadb **C:\Program Files\MariaDB 12.3\bin**
- Para poder tener un entorno visual que le permite crear tablas y verificar la creación de columnas puede utilizar [workbench](#),

Conectar la el proyecto con mysql

1. Django nos permite conectarnos con diferentes tipos de base de datos, por defecto el utiliza SQLite, que para pruebas rápidas está muy bien, nosotros nos conectaremos con mysql, pero para ello debemos instalar los drivers correspondientes
2. Para instalar los drivers de mysql los realizaremos con el comando **pip install mysqlclient**
3. Una vez instalado el driver procedemos a configurar la conectividad del proyecto con la base de datos para ello modificaremos el archivo > **comercial/settings.py**, de la siguiente manera

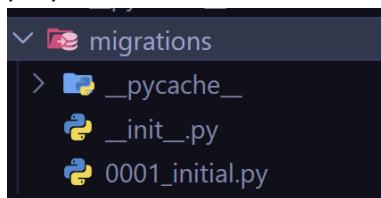
```

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'comercial',
        'USER': 'root',
        'PASSWORD': '',
        'HOST': 'localhost',
        'PORT': '3307'
    }
}

```

Migraciones

4. Una vez hemos configurado nuestra base de datos podemos crear las migraciones
5. Que son las migraciones, las migraciones es la preparacion que realizamos en cada cambio que realizamos en los modelos, tenga en cuenta que los modelos estaran enlazados con nuestras tablas en la base de datos
6. Para poder prepara las migraciones ejecutaremos el comando > **python .\manage.py makemigrations**
7. Lo enteriro le agera genera un nuevo archivo en la capeteta de migraciones corespondinete al proyecto



8. Si miramos el contenido del archivo 0001_initial.py, encontrar las instrucciones de creacion de la tablas que hemos generaado, pero cuado realicemos un cambio en dicha tablas agregare otro archivo con el cambio nuevo.

```
# Generated by Django 6.1.1 on 2026-09-14 17:52

from django.db import migrations, models

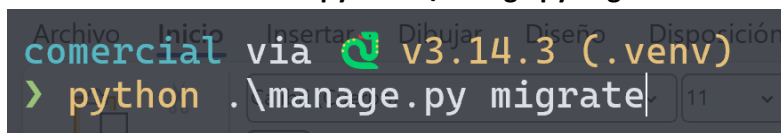
class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

    operations = [
        migrations.CreateModel(
            name='Categoria',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('nombre', models.TextField(max_length=255)),
                ('descripcion', models.TextField(max_length=255, null=True)),
                ('created_at', models.DateTimeField()),
                ('updated_at', models.DateTimeField()),
            ],
        ),
    ]
```

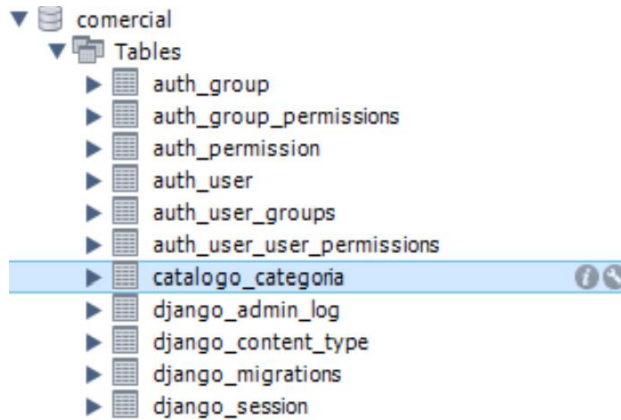
9. Para al fin crear la tabal en la base de datos deberemos de ejecutar las migraciones , esto lo aremos con el comando > **python .\manage.py migrate**



10. Si todo a salido bien , la consola le mostrar un mensaje como el siguiente

```
> python .\manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, catalogo, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial ... OK
  Applying auth.0001_initial ... OK
  Applying admin.0001_initial ... OK
  Applying admin.0002_logentry_remove_auto_add ... OK
  Applying admin.0003_logentry_add_action_flag_choices ... OK
  Applying contenttypes.0002_remove_content_type_name ... OK
  Applying auth.0002_alter_permission_name_max_length ... OK
  Applying auth.0003_alter_user_email_max_length ... OK
  Applying auth.0004_alter_user_username_opts ... OK
  Applying auth.0005_alter_user_last_login_null ... OK
  Applying auth.0006_require_contenttypes_0002 ... OK
  Applying auth.0007_alter_validators_add_error_messages ... OK
  Applying auth.0008_alter_user_username_max_length ... OK
  Applying auth.0009_alter_user_last_name_max_length ... OK
  Applying auth.0010_alter_group_name_max_length ... OK
  Applying auth.0011_update_proxy_permissions ... OK
  Applying auth.0012_alter_user_first_name_max_length ... OK
  Applying catalogo.0001_initial ... OK
  Applying sessions.0001_initial ... OK
```

11. Si visualiza la base de datos desde su gestor, podrá notar que se ha creado muchas tablas, pero entre ellas encontrará una tabla que tendrá de nombre el nombre de la app, catálogo más el nombre de la tabla categoría, así es como Django crea las tablas en la base de datos

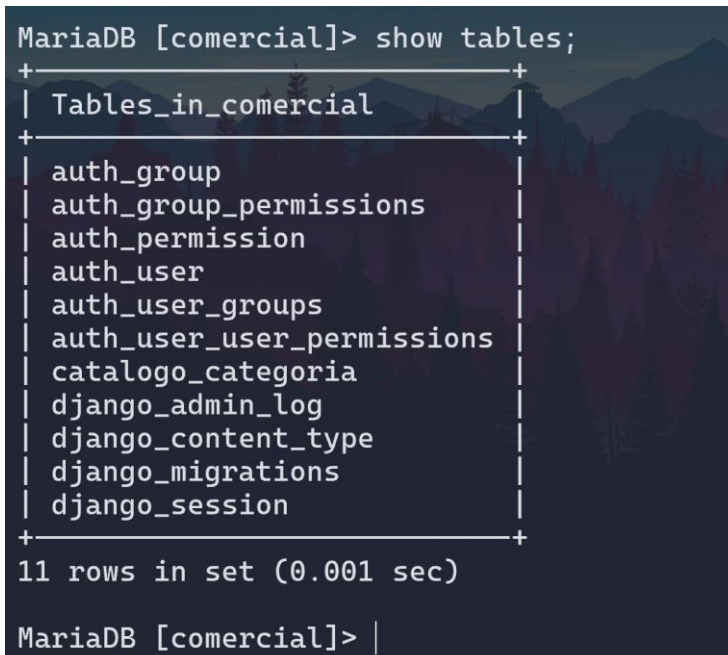


Revisión de la base de datos desde la consola interactiva

12. Django nos proporciona una consola interactiva con la base de datos, para poder utilizarla deberá de ejecutar el comando `>python .\manage.py dbshell`, esto nos permitirá ver las tablas de la base de datos desde la consola



13. Podemos visualizar las tablas con el comando de mysql `> show tables;`



14. si desea ver la estructura de creación de la tabla catalogo_categoria, puede utilizar el comando `> describe catalogo_categoria;`

```
MariaDB [comercial]> describe catalogo_categoria;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | bigint(20) | NO | PRI | NULL | auto_increment |
| nombre | longtext | NO | | NULL | |
| descripcion | longtext | YES | | NULL | |
| created_at | datetime(6) | NO | | NULL | |
| updated_at | datetime(6) | NO | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.013 sec)

MariaDB [comercial]> |
```

15. para poder salir de la consola interactiva de mariadb puede utilizar el comando **>exit;**

```
MariaDB [comercial]> exit;
```

Consularas

16. Django posee una herramienta que permite cargar todos los modelos en memoria para que podamos interactuar con la base de datos y realizar alguna pruebas de como mostrar , agregar , editar y eliminar datos
17. El comando que debera de utilizar es **> python .\manage.py shell**

Trabajando de la base de datos

18. En la consola de shell podremos utilizar lo mas parecido posible a como se relizara el codigo como se relizara mas adelante para agregar los reguistos en la base de datos

```
from django.utils import timezone
from catalogo.models.categoria import Categoria
```

```
mi_categoria = Categoria.objects.create(
    nombre="Electrónicos",
    descripcion="Productos electrónicos",
    created_at=timezone.now(),
    updated_at=timezone.now()
)
```

```
>>> from django.utils import timezone
>>> from catalogo.models.categoria import Categoria
>>> mi_categoria = Categoria.objects.create(
...     nombre="Electrónicos",
...     descripcion="Productos electrónicos",
...     created_at=timezone.now(),
...     updated_at=timezone.now()
... )
>>> |
```

19. para poder visualziar la inforacion de lo que hemos crado podemos utilizar el comando print , como se muestra a continuacion >> print (mi_categoria.)

```
>>> print(mi_categoria)
Categoría : Electrónicos Description: Productos electrónicos Fecha de creación : 2026-09-14 18:46:25.743094+00:00
0 Fecha de actualización 2026-09-14 18:46:25.743131+00:00
>>> |
```

Los anteriro es posible porque hemos crado la funcion __str__ dentro del modelo , si no estuviece solo se mostraria el mensaje object

20. lamentable mente el dato no esta reguistrado en la base de datos , todo lo anterior esta en la memoria ram , para poder agergarlo a la base de datos Django porvee de una funcion llamada save y se realizar de la siguiente manera >> mi_categoria.save()

```
>>> mi_categoria.save()
>>> |
```

21. con lo anterior tenemos nuetra primera categoria en la base de datos

editar los datos

22. ahora que ya sabemos como podemos agregar una categoira a la base de datos vermos como podemos moficar los datos

23. nos aprobecharmos que tenemos la instancia del objeto y modificaremos el nombre

```
>>mi_categoria.nombre ='tv'
```

```
>>> mi_categoria.nombre ='tv'
```

24. podemos imprimir los datos del objeto con el comando >> print (mi_categoria.) y podra notar que el datos a cambiado

```
>>> print(mi_categoria)
Categoría : tv Description: Productos electrónicos Fecha de creación : 2026-09-14 18:46:25.743094+00:00 Fecha de
actualizacion 2026-09-14 18:46:25.743131+00:00
```

25. todo lo anteriro no esta almacenado en la base de datos , y para podaer almacenarlo podemos utilizar el comano save , >> mi_categoria.save()

Eliminar datos en la base de datos

26. Django nos provee la facilidad de poder eliminar un reguistro que ya hayamos instanciado con el comando delete , para poder lo probar puede utilizar el comando >>> mi_categoria.delete()

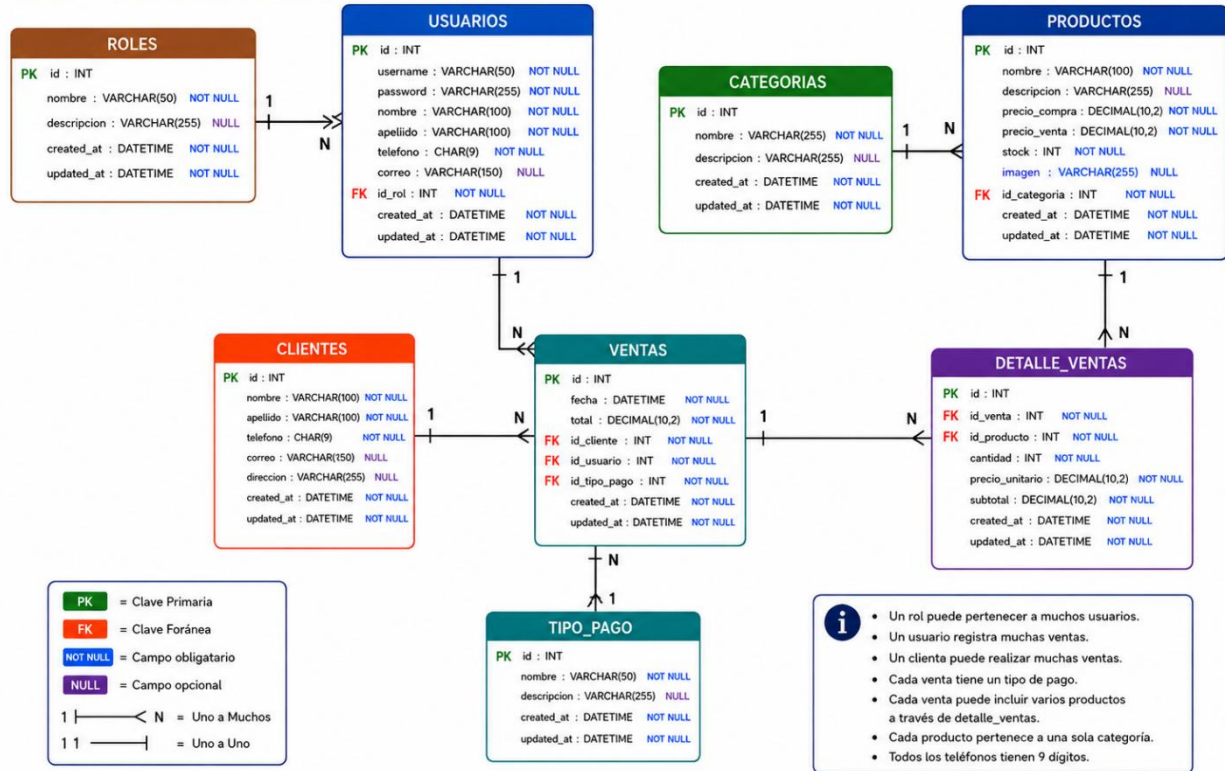
```
>>> mi_categoria.delete()
(1, {'catalogo.Categoria': 1})
>>> |
```

27. Lo anteriro regresara una tupla , donde el primer campo indica la cantidad de datos eliminados y el segundo el tido de datos eliminado



EJERCICIOS DE PRÁCTICA:

SISTEMA COMERCIAL



Genere los siguientes modelos y cree las migraciones correspondientes

1. Tipo_pago
2. Clientes
3. Roles